Sprites API v1

Examples: Python

Port Proxy

SPRITE-ENV v0.0.1-rc43

Tunnel TCP connections directly to services running inside your Sprite. After a brief WebSocket handshake, the connection becomes a transparent relay to any port.

Use this to access dev servers, databases, or any TCP service as if it were running locally. The proxy handles connection setup; your client speaks directly to the target service.

TCP Proxy

WSS `/v1/sprites/{name}/proxy`

Establish a WebSocket tunnel to a port inside the sprite environment.

JSON MESSAGES

After connecting, send a JSON init message to specify the target host and port.

`ProxyInitMessage` Client → Server

`host*` String
Target hostname (typically "localhost")

`port*` integer
Target port (1-65535)

BINARY PROTOCOL

After the JSON handshake completes, the connection becomes a raw TCP relay. Binary data is forwarded directly without any framing or prefixes.

NAME	DIRECTION	DESCRIPTION
data	client → server	Raw bytes sent to the target TCP port

Sprites API v1

Examples:

data server → client Raw bytes received from the target TCP port

RESPONSE CODES

101	Switching Protocols - WebSocket connection established
400	Bad Request - Invalid WebSocket upgrade or missing parameters
404	Not Found - Resource not found

bash *Python example unavailable* HTTP

```bash
websocat \
  "wss://api.sprites.dev/v1/sprites/{name}/proxy" \
  -H "Authorization: Bearer $SPRITES_TOKEN"
```

Protocol Flow

1. Send JSON init message
2. Receive JSON response
3. Binary TCP relay begins

JSON Messages

Init connection (client → server):

```
{"host": "localhost", "port": 8080}
```

Connection response (server → client):

```
{"status": "connected", "target": "localhost:8080"}
```